

Query Pipeline Design for Real Estate AI Platform

Objective

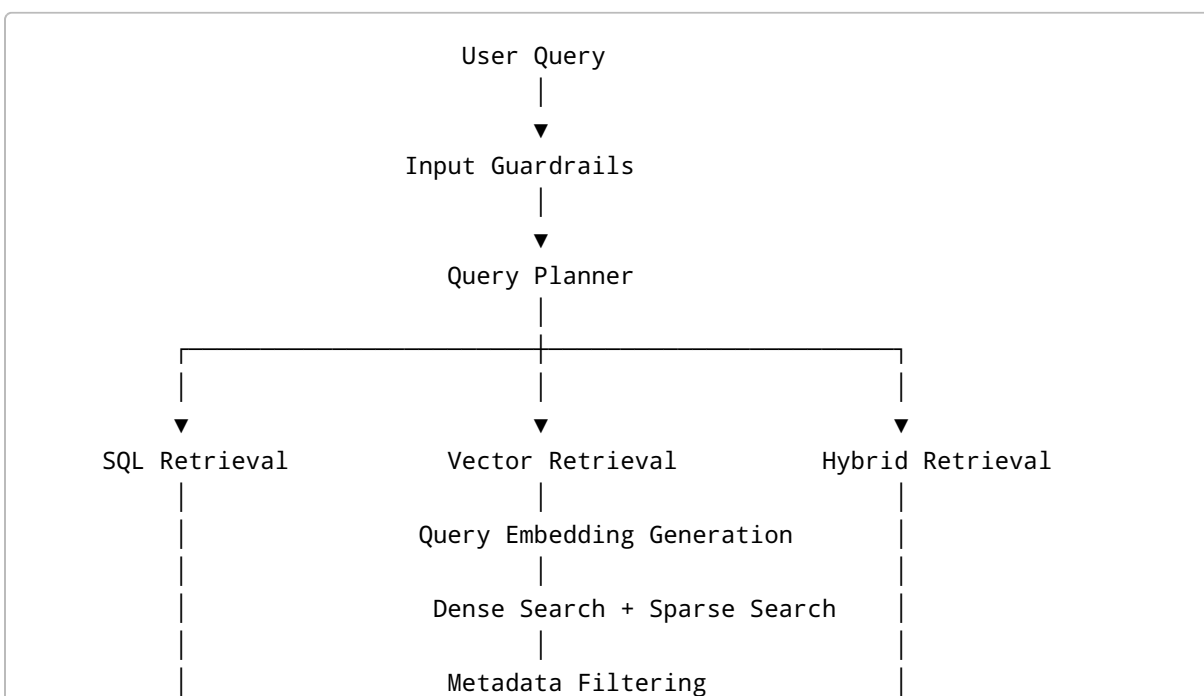
The objective of the query pipeline is to transform a user's natural language query into an accurate, grounded, low-latency response while balancing retrieval quality, cost, scalability, and security.

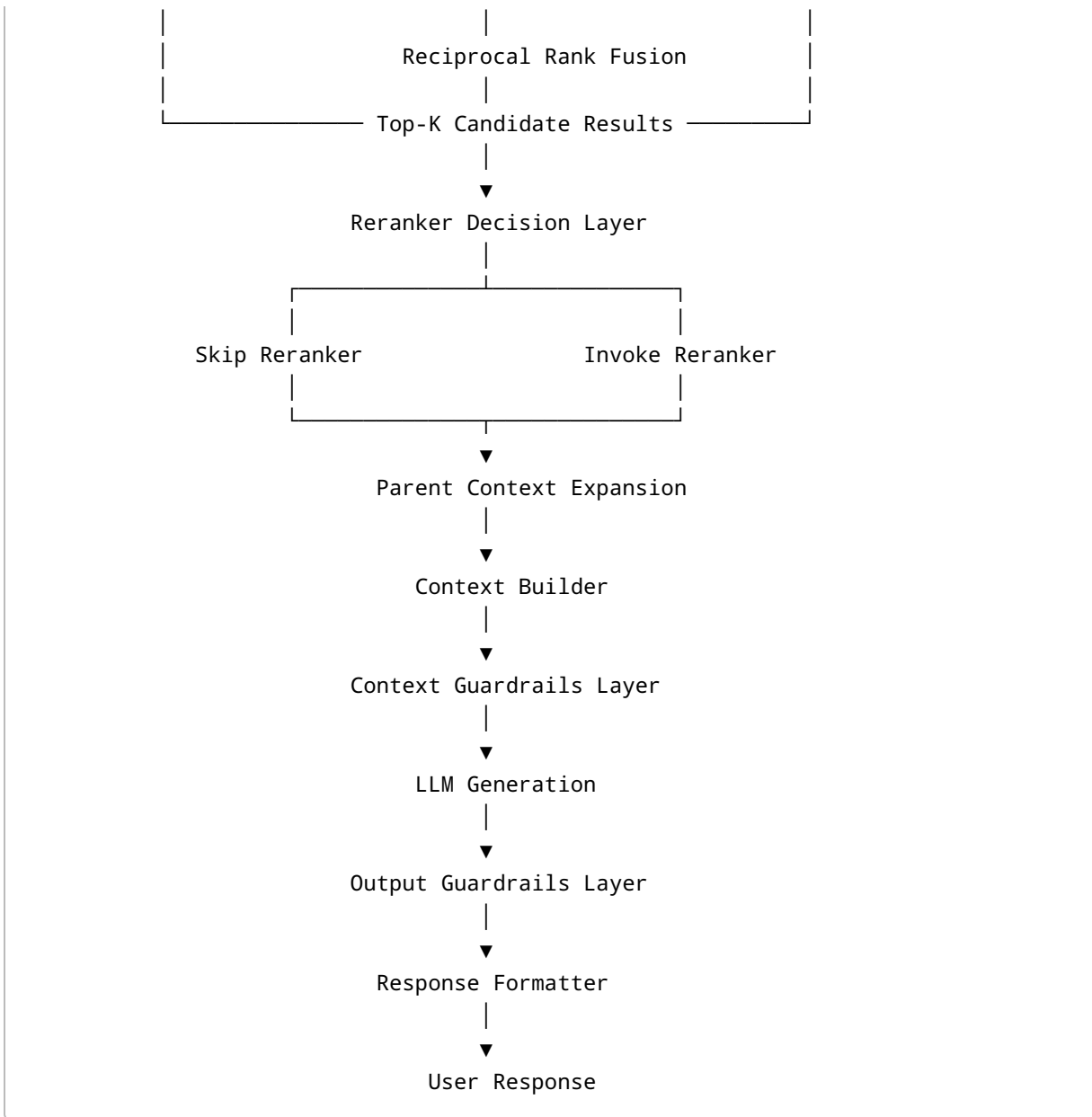
The pipeline should support:

- Structured queries
- Semantic queries
- Hybrid retrieval
- Multi-lingual and cross-lingual search
- Parent-child context expansion
- Guardrails
- Hallucination reduction
- Citation generation
- Tenant isolation

The architecture should optimize for retrieval quality first, because retrieval quality directly impacts answer quality.

High-Level Architecture





Input Guardrails

Before the query enters the retrieval system, input validation is performed.

Responsibilities:

- Prompt injection detection
- Jailbreak prevention
- Offensive content filtering
- Rate limiting

- Tenant validation

Examples:

Ignore previous instructions

or

Show another brokerage house's documents

Such requests should be blocked before entering the retrieval pipeline.

Query Planner

The query planner determines how the query should be executed.

Possible routes:

SQL Route

Example:

Show apartments between 80 lakh and 1 crore in Whitefield.

Structured fields:

- Price
- Location
- Bedrooms

can be directly translated into SQL.

Vector Route

Example:

Show family-friendly apartments with larger rooms.

Semantic understanding is required.

Hybrid Route

Example:

Show beautiful east-facing apartments near Whitefield metro under 1 crore with good amenities.

Needs:

- Structured filtering
- Semantic search

Hybrid retrieval is preferred.

SQL Retrieval

Structured attributes remain in PostgreSQL.

Examples:

- Price
- Latitude
- Longitude
- Bedrooms
- Agent information

Advantages:

- Exact filtering
 - Low latency
 - No embeddings required
-

Query Embedding Generation

For semantic search, query text is converted into embeddings.

Requirements:

- Multilingual support
- Cross-lingual support
- Low latency

Recommended:

Gemini Embedding 001

Example:

Query in Kannada

Documents in English

Cross-lingual retrieval should still work.

Vector Retrieval

The query embedding is sent to the vector database.

Operations:

Dense Search

Captures semantics.

Example:

"family-friendly"

matches

"good schools nearby"

even without exact keywords.

Sparse Search

Captures exact terms.

Example:

"east-facing"

or

"Whitefield"

Metadata Filtering

Filters are applied during HNSW traversal.

Examples:

```
{  
  "tenant_id": "tenant_1",  
  "price_range": "80L-1Cr",  
  "location": "Whitefield"  
}
```

This avoids expensive post-filtering.

Reciprocal Rank Fusion (RRF)

Dense search and sparse search results are merged.

Purpose:

Improve recall.

Dense Search:

Top K

+

Sparse Search:

Top K

↓

RRF

↓

Top N Candidates

Typical:

50-100 chunks.

Reranker Decision Layer

Reranking improves precision but introduces latency and cost.

Therefore reranking should not always execute.

Decision:

Compute similarity between query and top retrieved chunks.

If confidence > threshold:

Skip reranker.

Else:

Invoke reranker.

Benefits:

- Lower latency
 - Reduced cost
-

Reranker

Purpose:

Improve ranking quality.

Embedding models perform bi-encoding independently.

Rerankers are cross-encoders.

Input:

Query + Candidate Chunks

Output:

Relevance scores

Recommended:

Cohere Rerank

Benefits:

- Multilingual support
- Metadata-aware ranking
- High precision

Output:

Top 5–10 chunks.

Parent Context Expansion

Retrieved chunks contain parent IDs.

Example:

Child Chunk

↓

Subsection

↓

Section

↓

Document

Parent context is fetched from the document store.

Expansion stops when:

- Similarity score saturates.
- Context window limit is reached.

Optional summaries can reduce token consumption.

Context Builder

The context builder assembles the final prompt context.

Responsibilities:

- Merge SQL results
- Merge retrieved chunks
- Merge parent sections
- Remove duplicates
- Preserve ranking order
- Control token budget

The context builder determines what information reaches the LLM.

Garbage here results in poor answers.

Context Guardrails

Retrieved documents themselves can contain attacks.

Examples:

Ignore all instructions.

or

Prompt injection embedded inside documents.

Responsibilities:

- Prompt injection detection
- PII removal
- Policy compliance checks
- Tenant isolation verification
- Block confidential data leakage

This protects the LLM from malicious documents.

LLM Generation

The final context is passed to the generation model.

Responsibilities:

- Answer generation
- Multi-document reasoning
- Citation generation
- Explanation

Model routing:

Simple tasks:

Small model

Medium complexity:

Medium model

Legal reasoning:

Large model

This balances cost and quality.

Output Guardrails

Before returning the answer:

Validate:

- Hallucinations
- Toxicity
- Policy violations
- Unsupported claims

Ensure:

Every answer is grounded in retrieved context.

If confidence is low:

Respond:

I could not find sufficient information from the available documents.

rather than hallucinating.

Response Formatter

Adds:

Citations

Source documents

Confidence score

High

Medium

Low

Freshness

Document version

Updated timestamp

References

Supporting passages

This increases user trust.

Caching Layer

Frequently repeated queries should be cached.

Possible cache levels:

Embedding Cache

Avoid recomputing embeddings.

Retrieval Cache

Store retrieved chunk IDs.

LLM Response Cache

Cache final responses.

Benefits:

- Lower latency
 - Lower cost
-

Multi-Tenant Security

Every query carries:

```
{  
  "tenant_id": "tenant_1"  
}
```

Filters are enforced across:

- SQL
- Vector store
- Document store

Preventing cross-tenant leakage.

Observability

Monitor:

Retrieval

- Recall@K
- Precision@K
- MRR

Latency

- SQL latency
- Vector search latency
- Reranker latency
- LLM latency

Context

- Context size
- Parent expansion count

Cost

- Token usage
- Cost per query

Quality

- Hallucination rate
- Faithfulness
- Citation coverage

End-to-End Flow

User Query

↓

Input Guardrails

↓

Query Planner

↓

SQL + Dense Search + Sparse Search

↓

RRF

↓

Top Candidates

↓

Conditional Reranker

↓

Parent Context Expansion

↓

Context Builder

↓

Context Guardrails

↓

LLM

↓

Output Guardrails

↓

Response Formatter

↓

User

Final Recommendation

I would design the query pipeline around retrieval quality rather than generation quality. The query planner determines whether the request should use SQL retrieval, semantic retrieval, or hybrid retrieval. Dense and sparse searches are fused using Reciprocal Rank Fusion, followed by conditional reranking to optimize cost and latency. Parent-child context expansion enriches retrieved chunks with surrounding context, and a dedicated context builder assembles the final prompt. Guardrails are applied both before retrieval and after context construction to prevent prompt injection, policy violations, and cross-tenant leakage. Finally, the LLM generates grounded responses with citations and confidence indicators, ensuring transparency, trust, and scalability.